Apache Pig

November 3, 2025

Introduction:

Apache Pig is a high-level platform for processing large-scale data sets in the Hadoop ecosystem. Provides an abstraction layer over the complexities of MapReduce, allowing users to perform data manipulation and analysis. It uses a language called Pig Latin, which makes it easier to write data analysis programs compared to traditional MapReduce.

Key Features:

- Simplifies MapReduce -> Pig translates Pig Latin scripts into MapReduce jobs automatically.
- Handles structured & unstructured data.
- Extensible -> Users can write User Defined Functions (UDFs) in Java, Python, etc.
- Optimized execution -> Pig optimizes tasks internally for better performance.

Parallel Processing using Pig:

Pig is inherently parallel because it translates Pig Latin into a series of MapReduce jobs.

How Pig facilitates parallel processing:

- Compilation to MapReduce
- Automatic Parallelism
- Data Flow Optimization
- PARALLEL Clause
- Multi-Query Execution

Pig Architecture:

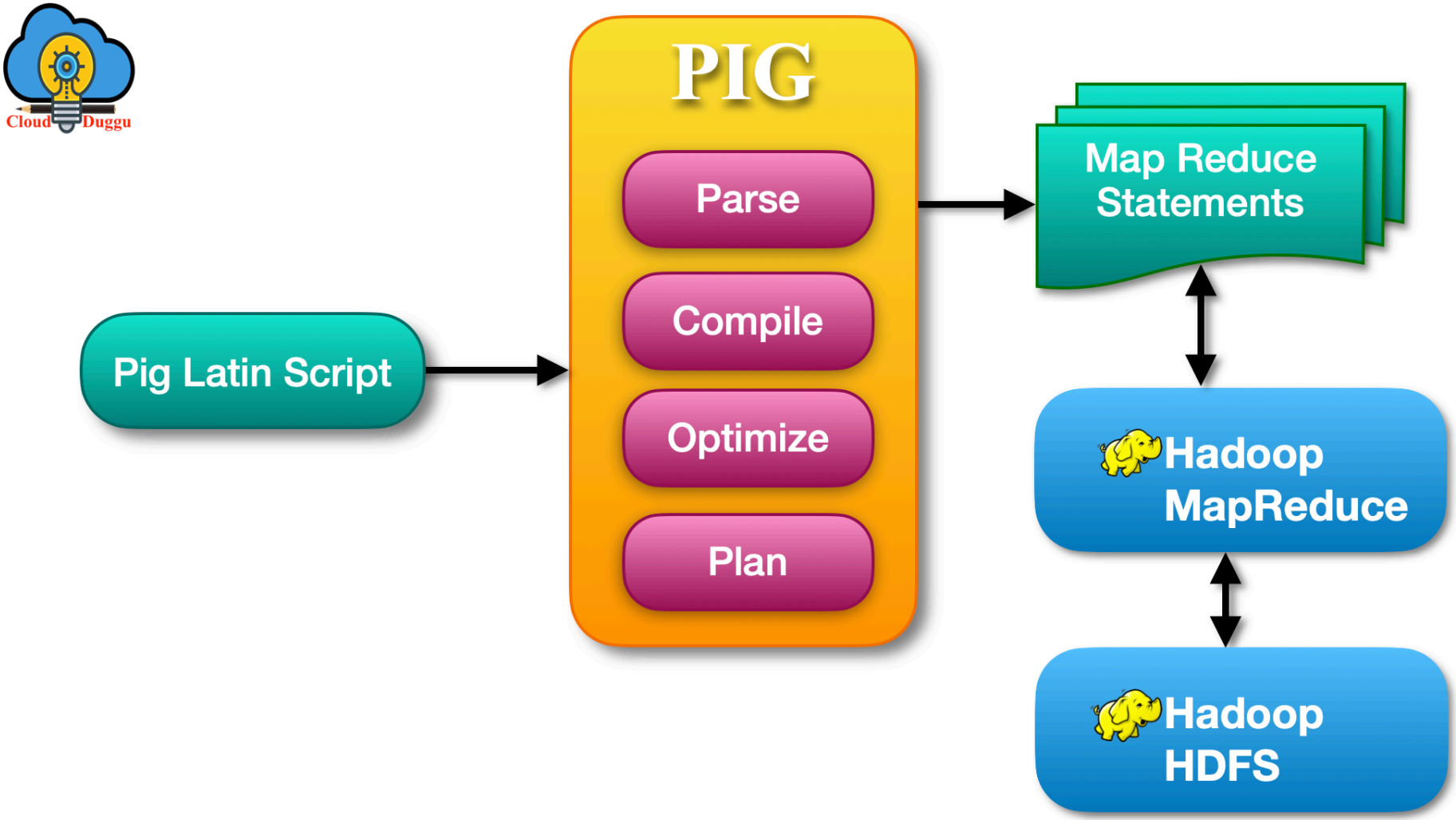

Figure: Pig Architecture

Pig Architecture:

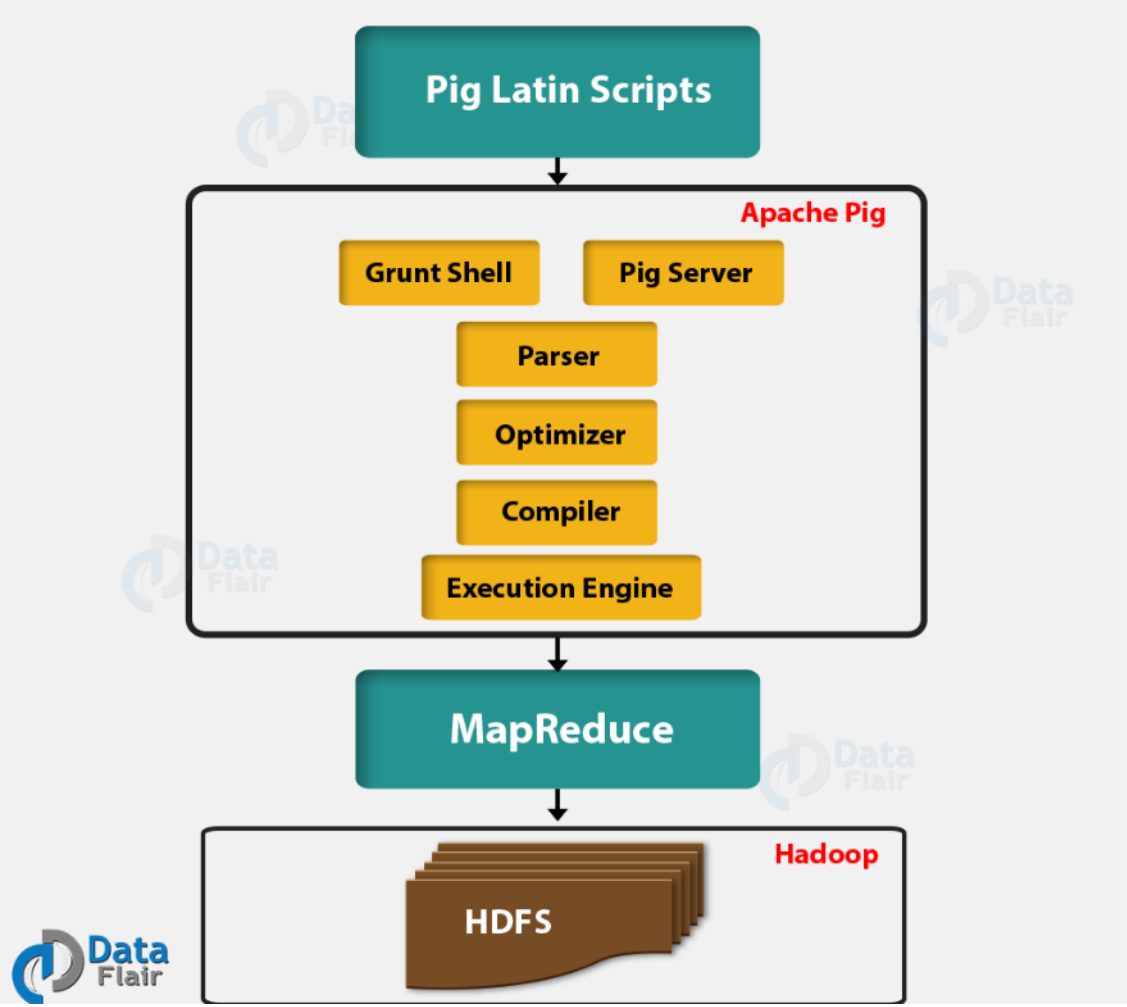

Figure: Pig Architecture

Apache Pig Components

Apache Pig process Latin programs through multiple layers.
The following are the component of Apache Pig.

- Parser:
  When a user submits a program, it is initially received by the parser, now the parser performs a syntax check and type checks. Once it performs this activity the output is created a DAG which contains Pig Latin statements and logical operators.
- Optimizer:
  In the optimization phase, the DAG is pushed to a logical optimizer that carries out logical optimizations such as projection and pushdown.
- Compiler:
  In the compiler phase, the complication of the optimized logical plan is done into small Mapreduce jobs and passed to the execution engine.
- Execution Engine:
  This is the final step in which Mapreduce jobs are submitted to Hadoop for execution. After completion of execution, it sends the desired data to the user.

Apache Pig Mode:

The following are three interaction modes allowed by Apache Pig.

- 1. Interactive Mode:
  In this mode of Apache Pig Interaction, the user uses Grunt shell to run Pig commands. The processing and complication are started when the user submits the STORE command.
- 2. Batch Mode:
  In this mode of Apache Pig interaction, a user runs a script file that contains a set of Pig command to perform some action.
- 3. Embedded Mode:
  This mode uses the Java library and the method invocation to run Apache Pig commands.

Apache Pig Grunt Shell:

Apache Pig Grunt is an interactive shell that enables users to enter Pig Latin interactively and provides a shell to interact with HDFS and local file system commands. You can enter Pig Latin commands directly into the Grunt shell for execution. Apache Pig starts executing the Pig Latin language when it receives the STORE or DUMP command. Before executing the command Pig Grunt shell do check the syntax and semantics to void any error.
To start Pig Grunt type :
$pig -x local
It will start Pig Grunt shell: grunt>

- grunt> fs -ls /
- grunt> fs -cat /hive/warehouse/kv2.txt
- grunt> fs -mkdir /pigdata
- grunt> fs -copyFromLocal /home/cloudduggu/pig/tutorial/emp.txt /pigdata/

Pig's Data Model:

Pig's data types can be divided into two categories: scalar types, which contain a single value, and complex types.

Scalar Types:

- int:
  An integer. Ints are represented in interfaces by java.lang.Integer. They store a four-byte signed integer. Constant integers are expressed as integer numbers, for example, 42.
- long:
  A long integer. Longs are represented in interfaces by java.lang.Long. They store an eight-byte signed integer. Constant longs are expressed as integer numbers with an L appended, for example, 5000000000L.
- float:
  A floating-point number. Floats are represented in interfaces by java.lang.Float and use four bytes to store their value.
  Complex Types (nested structures): Tuple, Bag, Map.

Pig Latin Input and Output:

Pig Latin is the language used in Apache Pig to work with Big Data. To do any work, Pig needs two things:

1. Input
2. Output

1. Input in Pig Latin: Input means reading data into Pig.

We use the LOAD statement.

Syntax:

relation = *LOAD*′*file*$_{l}$*ocation*′ USING function AS schema; Example:

students = LOAD 'hdfs:/user/data/students.txt' USING PigStorage(',') AS (id:int, name:chararray, marks:int);

2. Output in Pig Latin: After processing data, we want to see or save the results.

Two commands are used:

DUMP → shows result on the screen

STORE → saves result to a file

Relational Operators:

Operator	**Purpose**
LOAD	Load data into Pig
FOREACH	Select/transform columns
FILTER	Select rows based on condition
GROUP	Group rows by column
ORDER	Sort rows
DISTINCT	Remove duplicates
JOIN	Combine relations
CROSS	Cartesian product
LIMIT	Restrict number of rows
UNION	Merge relations
SPLIT	Divide relation into parts

Table: Relational Operators in Pig

User Define Functions:

Types of UDF (User Defined Functions) in Pig:
Eval Functions $\rightarrow$ process input and return output (most common).
Example: converting strings to uppercase.
Filter Functions $\rightarrow$ return true/false to decide whether a record should be included.
Load/Store Functions $\rightarrow$ define custom ways to read/write data.
Algebraic Functions $\rightarrow$ special case of eval functions that work on grouped data

Using the UDF in Pig Script:

Step 1: Register the jar REGISTER *'upper.jar'*;
Step 2: Define alias for the UDF DEFINE ToUpper Upper();
Step 3: Load data data = *LOAD' hdfs* : */user/data/names.txt'* USING PigStorage(',') AS (name:chararray);
Step 4: Apply UDF upper data = FOREACH data GENERATE ToUpper(name);
Step 5: Dump result DUMP upper data;